

1. PERSIAPAN & IMPORT PUSTAKA

Sebelum memulai analisis data, lakukan import pandas dan numpy (opsional namun sering dibutuhkan):

```
# Import pustaka standar
import pandas as pd
import numpy as np
```

2. MEMBACA & MENYIMPAN DATA (I/O)

FUNGSI FUNGSI PEMBACAAN (IMPORT)	FUNGSI FUNGSI PENYIMPANAN (EXPORT)
pd.read_csv('file.csv') Membaca file CSV menjadi DataFrame.	df.to_csv('output.csv', index=False) Menyimpan ke CSV tanpa menulis indeks baris.
pd.read_excel('file.xlsx') Membaca file spreadsheet Excel.	df.to_excel('output.xlsx', index=False) Menyimpan DataFrame ke bentuk file Excel.
pd.read_json('file.json') Membaca data terstruktur format JSON.	df.to_json('output.json') Mengonversi dan menyimpan data ke format JSON.
pd.read_sql(query, connection) Membaca hasil query SQL dari database.	df.to_sql('tabel', connection) Menuliskan data langsung ke tabel database SQL.

3. MEMBUAT OBJEK DATA SECARA MANUAL

Membuat Series (1 Dimensi):

```
s = pd.Series([1, 3, 5, np.nan, 6, 8],
              index=['a', 'b', 'c', 'd', 'e', 'f'])
```

Membuat DataFrame (2 Dimensi) dari Dict:

```
data = {
    'Nama': ['Andi', 'Budi', 'Cici'],
    'Usia': [23, 28, 21],
    'Kota': ['JKT', 'BDG', 'SUB']
}
df = pd.DataFrame(data)
```

4. INSPEKSI & EKSPLORASI DATA AWAL

METODE / ATRIBUT	DESKRIPSI & KEGUNAAN
df.head(n) / df.tail(n)	Melihat n baris pertama atau n baris terakhir data (default: 5 baris).
df.shape	Mengembalikan tuple ukuran data berbentuk (jumlah_baris, jumlah_kolom).
df.info()	Menampilkan ringkasan struktur DataFrame: nama kolom, tipe data, jumlah non-null, memori.
df.describe()	Menghasilkan statistik deskriptif otomatis (mean, min, max, kuartil) untuk kolom numerik.
df.columns / df.index	Mengakses daftar nama seluruh kolom atau indeks baris DataFrame.
df['Kolom'].value_counts()	Menghitung frekuensi kemunculan setiap nilai unik pada kolom tertentu.
df['Kolom'].unique() / nunique()	unique() mengambil daftar nilai unik; nunique() menghitung jumlah nilai uniknya.

5. MEMILIH (SELECTION) & MEMFILTER DATA

```
# 1. Memilih Kolom Tertentu
df['Nama']          # Menghasilkan Series dari kolom 'Nama'
df[['Nama', 'Kota']] # Menghasilkan DataFrame berisi beberapa kolom

# 2. Memilih Berdasarkan Indeks Posisi (iloc) atau Label (loc)
df.iloc[0]          # Baris pertama (indeks posisi ke-0)
df.iloc[0:5, 1:3]    # Slicing: baris 0 s.d 4, kolom 1 s.d 2
df.loc[df.index[0]]  # Baris berdasarkan label indeks tertentu
df.loc[:, 'Nama':'Kota'] # Mengambil semua baris dari rentang kolom 'Nama' hingga 'Kota'

# 3. Memfilter Baris Berdasarkan Kondisi (Boolean Indexing)
df[df['Usia'] > 22]    # Mengambil baris dengan nilai Usia di atas 22
df[(df['Usia'] > 20) & (df['Kota'] == 'JKT')] # Logika AND (&) membutuhkan tanda kurung
df[(df['Kota'] == 'BDG') | (df['Kota'] == 'SUB')] # Logika OR (|)
df[df['Kota'].isin(['JKT', 'BDG'])] # Menggunakan isin() untuk mencocokkan beberapa opsi
```

6. PEMBERSIHAN DATA (DATA CLEANING)

SINTAKS / PERINTAH	PENJELASAN FUNGSI
<code>df.isna().sum()</code> atau <code>df.isnull().sum()</code>	Menghitung jumlah nilai kosong (missing values/NaN) di setiap kolom.
<code>df.dropna()</code>	Menghapus seluruh baris yang mengandung minimal satu nilai NaN.
<code>df.dropna(axis=1)</code>	Menghapus seluruh kolom yang mengandung nilai NaN.
<code>df.fillna(value)</code>	Mengisi semua nilai NaN dengan nilai pengganti (misal: angka 0 atau nilai mean).
<code>df['Kolom'].fillna(df['Kolom'].mean())</code>	Imputasi nilai kosong spesifik kolom menggunakan rata-rata kolom tersebut.
<code>df.drop(['KolomA'], axis=1)</code>	Menghapus kolom 'KolomA' dari DataFrame.
<code>df.rename(columns={'Lama': 'Baru'})</code>	Mengubah nama kolom tertentu dari label 'Lama' menjadi 'Baru'.
<code>df.drop_duplicates()</code>	Menghapus baris-baris data yang terdeteksi duplikat penuh.
<code>df['Kolom'].astype('float')</code>	Mengubah/konversi tipe data kolom menjadi bentuk float (atau tipe data lain).

7. MANIPULASI, TRANSFORMASI & AGREGASI DATA

```
# 1. Mengurutkan Data (Sorting)
df.sort_values(by='Usia', ascending=True) # Urut berdasarkan Usia terkecil ke terbesar

# 2. Pengelompokan & Agregasi (Group By)
df.groupby('Kota')['Usia'].mean()          # Rata-rata Usia per kelompok Kota
df.groupby('Kota').agg({'Usia': ['mean', 'min', 'max'], 'Nama': 'count'}) # Multi-agregasi

# 3. Menerapkan Fungsi ke Data (Apply)
df['Nama_Upper'] = df['Nama'].apply(lambda x: x.upper()) # Transformasi kustom dengan lambda

# 4. Membuat Tabel Silang / Pivot
df.pivot_table(index='Kota', columns='Gender', values='Usia', aggfunc='mean')
```

8. PENGGABUNGAN & KOMBINASI DATA (MERGE / CONCAT)

METODE KOMBINASI	CONTOH KODE & CARA KERJA
Concatenation (Penggabungan) Menumpuk data secara langsung.	<code>pd.concat([df1, df2], axis=0)</code> # <i>Menumpuk baris kebawah</i> <code>pd.concat([df1, df2], axis=1)</code> # <i>Menyambung kolom kesamping</i>
Merging (SQL-like Join) Menggabungkan data berdasarkan kolom kunci (*key column*).	<code>pd.merge(df1, df2, on='ID', how='inner')</code> Metode how dapat diisi: 'inner', 'left', 'right', atau 'outer'.